

Moderné prístupy v optimalizácii

Teória optimalizačných úloh

prednáška 2

Ing. Ján Magyar, PhD.
ak. rok. 2026/2027 ZS

Od kandidátov k prehľadávaniu

kandidát = možné nastavenie komponentov

množina všetkých kandidátov = priestor kandidátov

priestor kandidátov môže byť spojitý alebo diskretný

analýza priestoru kandidátov udáva spôsob prehľadávania

Priestor prehľadávania

nemusí byť totožný s priestorom kandidátov

netreba prehľadávať celý priestor kandidátov

využijeme vhodnejšiu reprezentáciu

nie je dostupný efektívny algoritmus pre priestor kandidátov

odstránime duplicitné hodnoty

ak použijeme mapovanie do priestoru prehľadávania, je potrebná aj spätná transformácia

Plocha vhodnosti (fitness landscape)

ako algoritmus vidí priestor prehľadávania

vyjadruje hodnotu kandidáta (+1 dimenzia)

rôzne reprezentácie dajú rôzne plochy vhodnosti

Vlastnosti plochy vhodnosti

unimodálny vs multimodálny

(lokálne a globálne optimá)

hladkosť = ako veľmi sú si podobné hodnoty susediacich kandidátov

problematické sú aj plošiny aj prudké zmeny

Iteračné vylepšovanie

lokálne prehľadávanie

zadefinované okolie

na základe vzdialenosti

vymenovaním (explicitne alebo mapovaním)

Komponenty lokálneho prehľadávania

algoritmus lokálneho prehľadávania pre inštanciu π vyžaduje komponenty:

- $S(\pi), N(\pi), M(\pi)$
- inicializačná funkcia $init(\pi): \square \rightarrow D(S(\pi) \times M(\pi))$
- kroková funkcia $step(\pi): S(\pi) \times M(\pi) \rightarrow D(S(\pi) \times M(\pi))$
- ukončovací predikát $term(\pi): S(\pi) \times M(\pi) \rightarrow D(\{\top, \perp\})$

Komponenty iteračného vylepšovania

Funkcia *init* (Uniform Random Picking)

$$\textit{init}: p(x) = 1 / \#S \quad \textit{pre všetky } x \in S$$

Funkcia *step*

$$I(s) = \{ x \in N(s) \mid g(x) > g(s) \}$$

$$\textit{step}: \quad p(x) = 1 / \#I(s) \quad \textit{ak } x \in I(s)$$

$$p(x) = 0 \quad \textit{inak}$$

maximalizácia (existuje aj minimalizačná podoba)

Predikát *term*

$$\#I(s) = 0 \quad \textit{dosiahnutie lokálneho optima}$$

Algoritmus iteračného vylepšovania

input: π

output: $s \in S$ ☐

$s = urp()$

while($\#I(s) > 0$)

$s = select(I(s))$

endwhile

if($valid(s)$) **then**

return s

else

return ☐

endif

Uviaznutie v lokálnom optime

pridáme pamäť, okrem toho vieme použiť niekoľko prístupov:

- opakované reštarty

- zväčšenie okolia

- prepínanie okolí rôznych veľkostí

- akceptácia horšieho riešenia

- dynamická penalizácia

Algoritmus Jaya

opakované reštarty pokrývajú priestor v čase, Jaya má horizontálne škálovanie

populačný algoritmus = máme populáciu aktuálnych kandidátov

aktualizácia kandidátov = posun bližšie k najlepšiemu kandidátovi a ďalej od najhoršieho kandidáta

Aktualizácia kandidátov

$$x' = x + r_1(x_{best} - x) - r_2(x_{worst} - x)$$

$$r_1, r_2 \sim U(0, 1)$$

v skutočnosti máme niekoľko komponentov:

$$x'_{i,j} = x_{i,j} + r_{1,j}(x_{best,j} - x_{i,j}) - r_{2,j}(x_{worst,j} - x_{i,j})$$

akceptujeme iba lepších kandidátov:

$$f(x'_i) < f(x_i)$$

Algoritmus Jaya

1. inicializuj parametre
2. inicializuj populáciu
3. kým nie je splnená ukončovacia podmienka, urob:
 - 3.1. nájdi najlepšieho a najhoršieho kandidáta
 - 3.2. posuň kandidátov
 - 3.3. ak je modifikovaný kandidát lepší, prepíš ho, inak ponechaj starého kandidáta
4. vráť najlepšie nájdené riešenie

Výhody algoritmu Jaya

málo parametrov

nepotrebujeme poznať gradient

explorácia aj exploatácia

konvergencia k najlepšiemu riešeniu

Pri riešení maximalizačnej úlohy nám jeden algoritmus vrátil kandidáta s hodnotou cieľovej funkcie 0,9, druhý algoritmus našiel kandidáta s hodnotou 0,4. Ktorý z nich je lepší?

Kritériá hodnotenia algoritmov

kvalita výsledkov

výpočtová zložitosť

rýchlosť konvergenzie

robustnosť a spoľahlivosť

No Free Lunch teoréma

ak spriemerujeme výkonnosť optimalizačných algoritmov naprieč všetkými možnými cieľovými funkciami, priemerná výkonnosť je rovnaká

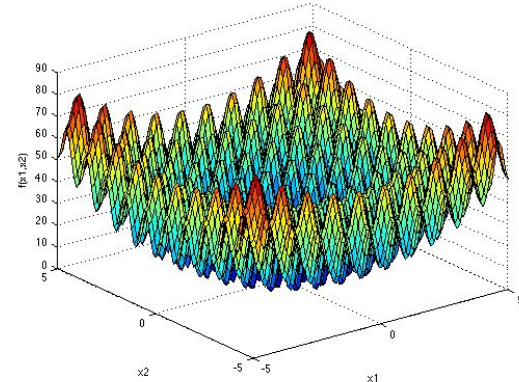
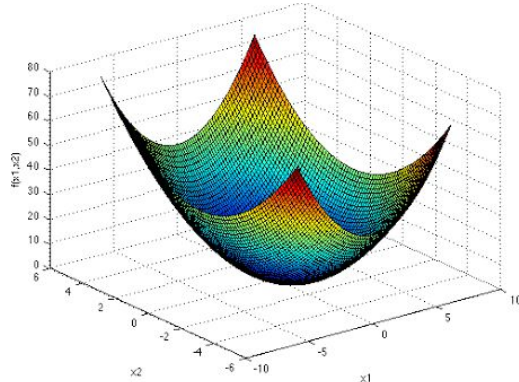
neexistuje univerzálne najlepší optimalizačný algoritmus

Benchmarky

optimalizačné algoritmy sa vyhodnocujú na reprezentatívnej vzorke optimalizačných problémov

pri porovnávaní algoritmov dodržíjeme rovnaké limity

často používané benchmark funkcie, napr. Sphere, Rastrigin



Zhrnutie

priestor kandidátov, priestor prehl'adávania, plocha vhodnosti

iteračné vylepšovanie

algoritmus Jaya

hodnotenie algoritmov